COMP7710
Programming Fundamentals

Today's lecture

Recursion

Course Convenor
Bernardo Pereira Nunes

Australian National University

All lectures are recorded and made available on Canvas

Agenda

> Recursion
> Examples
> Exercises

Recursion

"In order to understand recursion, one must first understand recursion"

> A **recursive function** is one that calls itself

Is it the same problem?
It reduces the problem into smaller/simpler versions of the same problem

>> Recursive functions include:

recursive case: a **call to the recursive function itself**

base case: the condition that **stops the function from calling itself**; it represents the simplest case, where the answer can be determined directly

Recursion makes problems easier to solve by breaking them into smaller sub-problems, thereby making the overall problem **easier to understand**

Examples

> Fibonacci sequence, Factorials, Tree traversals, …

Recursion: Factorial Example

> A factorial is a mathematical operation that multiplies a number by all the positive integers smaller than it:

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$$

> Instead of solving the whole problem at once, recursion breaks it into smaller versions of the same problem

$$n! = n \cdot (n-1)!$$
$$5! = 5 \cdot 4!$$

$$n! = \begin{cases} 1, & if\ n = 0 \\ n \cdot (n-1)! & if\ n > 0 \end{cases}$$

base case

recursive case

Recursion: Factorial Example

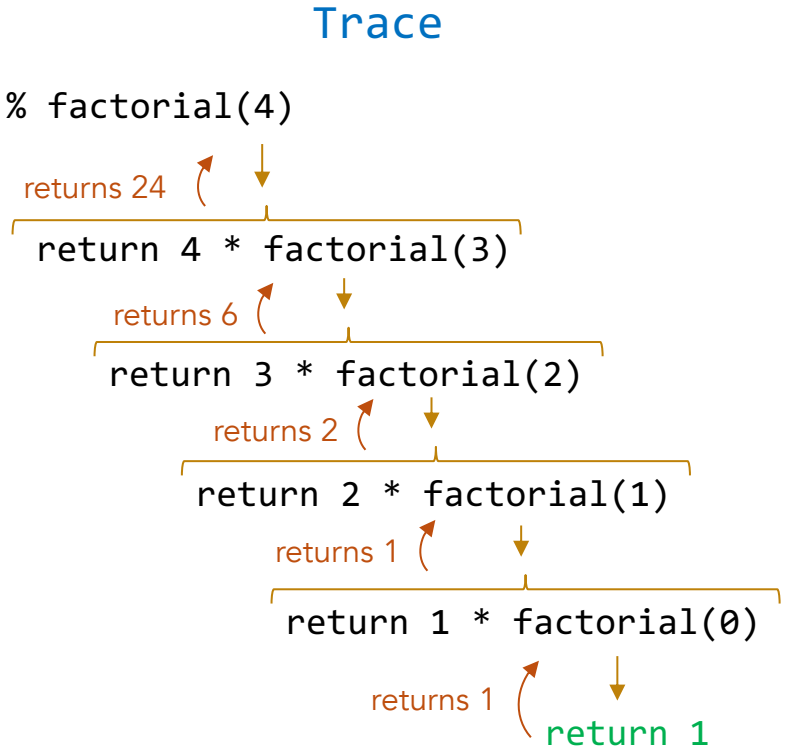
$$n! = \begin{cases} 1, & if\ n = 0 \\ n \cdot (n-1)!\ & if\ n > 0 \end{cases}$$

base case

recursive case

```
% public static int factorial(int n) {
      // Base case
      if (n == 0) {
          return 1;
      }

      // Recursive case
      return n * factorial(n - 1);
  }
```



Recursion: Factorial Example

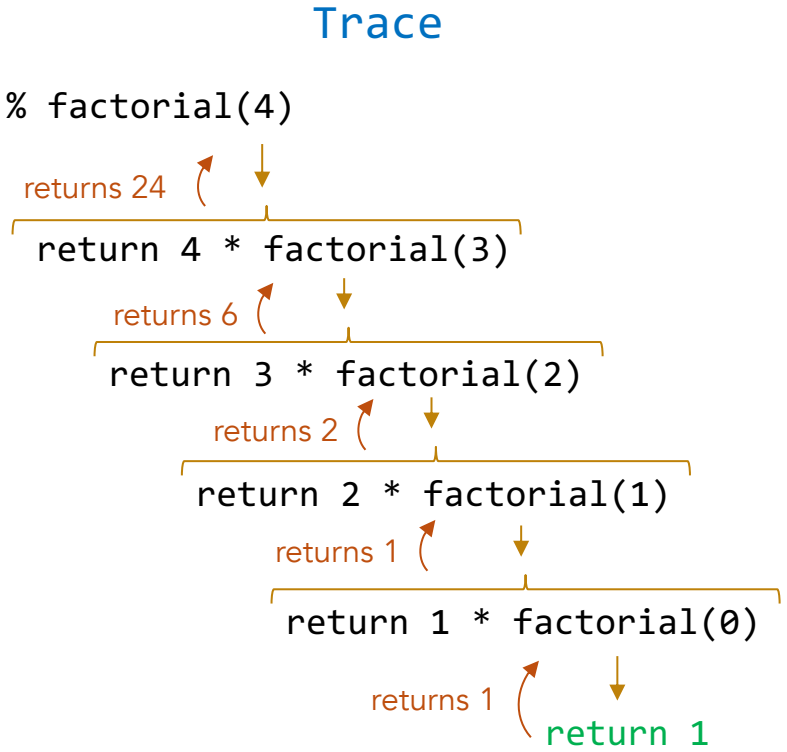
$$n! = \begin{cases} 1, & if\ n = 0 \\ n \cdot (n-1)!, & if\ n > 0 \end{cases}$$

base case

recursive case

```
% public static int factorial(int n) {
      // Base case
      if (n == 0) {
          return 1;
      }

      // Recursive case
      return n * factorial(n - 1);
  }
```



Recursion: Count Uppercase Chars

```
% public static int countUppercaseChars(String s) {
    // Base case
    if (s == null || s.length() == 0) {
        return 0;
    }

    // Recursive case
    if (s.charAt(0) >= 'A' && s.charAt(0) <= 'Z') {
        return 1 + countUppercaseChars(s.substring(1));
    } else {
        return countUppercaseChars(s.substring(1));
    }
}
```

base case

recursive case

returns a new string containing all characters from the original string starting from the index 1 through to the end

Recursion: Count Uppercase Chars

What is the difference between this and the previous recursive function?

```
% public static int countUppercaseChars(String s) {
    return countUppercaseChars(s, 0);
}
private static int countUppercaseChars(String s, int index) {
    // Base case
    if (s != null || index == s.length()) {
        return 0;
    }
    // Recursive case
    if (Character.isUpperCase(s.charAt(index))) {
        return 1 + countUppercaseChars(s, index + 1);
    } else {
        return countUppercaseChars(s, index + 1);
    }
}
```

base case

recursive case

it avoids creating new strings with `substring()`; it is more memory efficient; it uses `Character.isUpperCase()` which is clearer and works beyond just A-Z ASCII characters.

Russian Nested Dolls - Matryoshka

How many dolls in a Russian doll set?

Russian Nested Dolls - Matryoshka

How many dolls in a Russian doll set?

```
def howManyDolls(this_doll) // pseudo code
{
    if this_doll is empty                          base case
       return 1
    else                                           recursive case
       return 1 + howManyDolls(smaller_doll_inside);
}
```

Russian Nested Dolls - Matryoshka

How to create an OO version of the pseudo code?

```
public class MatryoshkaDoll {

    private final String name;
    private final MatryoshkaDoll innerDoll;

    private MatryoshkaDoll(String name, MatryoshkaDoll innerDoll) {
        this.name = name;
        this.innerDoll = innerDoll;
    }

...
}
```

Russian Nested Dolls - Matryoshka

How to create an OO version of the pseudo code?

```
...
    // Factory: creates the innermost doll with nothing inside
    public static MatryoshkaDoll of(String name) {
        return new MatryoshkaDoll(name, null);
    }

    // Factory: creates a doll that contains another doll inside
    public static MatryoshkaDoll of(String name, MatryoshkaDoll innerDoll) {
        return new MatryoshkaDoll(name, innerDoll);
    }

    // Recursively counts the total number of dolls, including this one and all nested dolls within it
    public int count() {
        if (innerDoll == null) return 1; // base case (no inner doll, just this doll)
        return 1 + innerDoll.count(); // recursive case (has inner doll, 1 + count of the inner doll)
    }

    public String getName() { return name; }
    public MatryoshkaDoll getInnerDoll(){ return innerDoll; }

    @Override
    public String toString() {
        if (innerDoll == null) return name;
        return name + " has " + innerDoll;
    }
```

Russian Nested Dolls - Matryoshka

How to create an OO version of the pseudo code?

```
// main call
MatryoshkaDoll set = MatryoshkaDoll.of("Doll 1",
                                       MatryoshkaDoll.of("Doll 2",
                                       MatryoshkaDoll.of("Doll 3",
                                       MatryoshkaDoll.of("Doll 4",
                                       MatryoshkaDoll.of("Doll 5")))));
```

There are many other ways to implement (for example using Java.util.Optional, Generics and streams)

Recursion

> When a function is called recursively, a local environment is created for each call

> The local variables of recursive calls are independent of each other, as if we were calling different functions

```
static void countdown(int n) {
        // Each call gets its OWN local copy of 'n'
        IO.println("Entering call  -> n = " + n);

        if (n == 0) {
            IO.println("Base case reached!");
            return;
        }

        countdown(n - 1);  // new call, new local environment

        // When we return here, OUR 'n' is still intact
        IO.println("Returning to   -> n = " + n);
    }
```

```
jshell> countdown(3)
Entering call  -> n = 3
Entering call  -> n = 2
Entering call  -> n = 1
Entering call  -> n = 0
Base case reached!
Returning to   -> n = 1
Returning to   -> n = 2
Returning to   -> n = 3
```

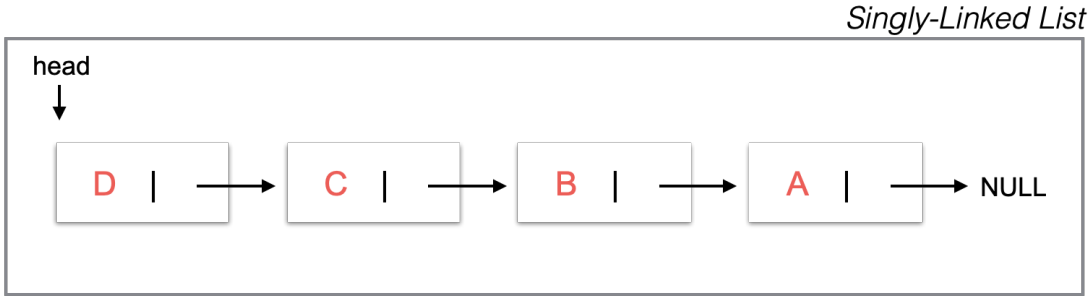
Recursion: LinkedList

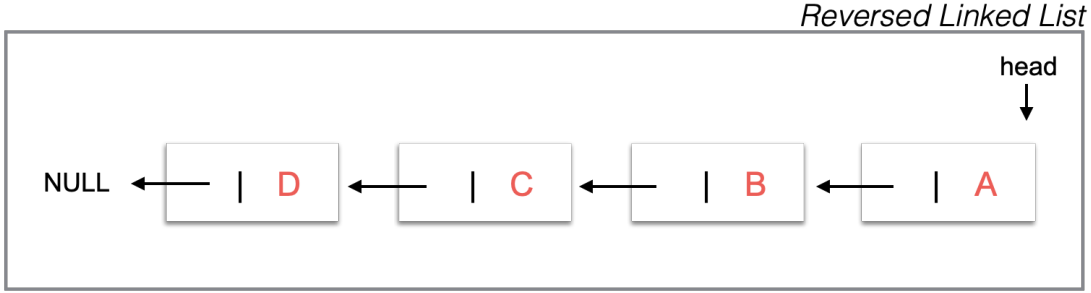
> Recursive definition of a LinkedList

>> A Linked List is either:

Recursion: LinkedList

> How can a singly-linked list be reversed by recursive reversal?





Recursion: LinkedList

> How can a singly-linked list be reversed by recursive reversal?

Recursion: *Break the problem into smaller sub-problems (list in sublists)*

Base Case

List is *empty*

List *has only one* node

Recursive Case

Pass the *next node* of the *current node* to the *reverse function*

Once the last node is reached, the reversing occurs

Recursion: LinkedList

> How can a singly-linked list be reversed by recursive reversal?

```
% static Node reverse(Node p) { // mutable version
        // Base case: empty list or single node: nothing to reverse
        if (p == null || p.next == null)
            return p;

        Node r = reverse(p.next); // Recursive case: reverse the rest of the list

        // swap/reversing
        Node q = p.next;  // q points to the node after p
        q.next = p;       // make that node point back to p
        p.next = null;    // detach p from the old next

        return r; // r is the new head (never changes)
    }
```


<Exercises>

What is most likely to happen if a recursive function has no base case?

A. The program runs faster
B. The recursion stops automatically
C. Infinite recursion may occur leading to a stack overflow
D. The compiler fixes the problem automatically

<Exercises>

Why is recursion useful?

A. It removes the need for conditions
B. It avoids using memory
C. It breaks a problem into smaller versions of the same problem
D. It replaces all loops

<Exercises>

Consider:

```
static void countdown(int n) {
    if (n == 0) {
        return;
    }
    countdown(n - 1);
}
```

What is the base case?

A. `countdown(n - 1)`
B. `if (n == 0)`
C. `static void countdown(int n)`
D. `return;`

<Exercises>

```
public static int mystery(int n) {
    if (n == 1) {
        return 1;
    }

    return n + mystery(n - 1);
}
```

What is returned by mystery(4)?

A. 4
B. 6
C. 10
D. 24

<Exercises>

What does this function do? Identify the base case(s) and recursive case(s).

```
public static int s(int n) {
    if (n == 0) {
        return 0;
    }

    return n + s(n - 1);
}
```

<Exercises>

What does this function do? Identify the base case(s) and recursive case(s).

```
public static int s(int n) {
    if (n == 0) {
        return 0;
    }

    return (n % 10) + s(n / 10);
}
```

<Exercises>

reverse("cat") = reverse("at") + "c”
Input: cat
Output: tac

Complete the following function with the base case:

```
public static String reverse(String s) {
    // Base case
    // TODO

    // Recursive case
    return reverse(s.substring(1)) + s.charAt(0);
}
```

<Exercises>

Complete the recursive case to check whether a String is a palindrome:

A palindrome reads the same forward and backward: level, racecar

```
public static boolean isPalindrome(String s) {
    // Base case
    if (s.length() <= 1) {
        return true;
    }

    // Characters do not match
    if (s.charAt(0) != s.charAt(s.length() - 1)) {
        return false;
    }

    // Recursive case
    return // TODO
}
```

<Exercises>

Binary Search (Recursive): Recursively search only half of the array

Create the call and explain step by step what is happening during the search

```
public static int binarySearch(int[] arr, int target, int left, int right) {
    // Base case
    if (left > right) {
        return -1;
    }

    int mid = (left + right) / 2;

    if (arr[mid] == target) {
        return mid;
    }

    // Recursive case
    if (target < arr[mid]) {
        return binarySearch(arr, target, left, mid - 1);
    } else {
        return binarySearch(arr, target, mid + 1, right);
    }
}
```

<Exercises>

Count nodes in a LinkedList

Create the call and explain step by step what is happening during the search

```
public static int countNodes(Node head) {

    // Base case
    if (head == null) {
        return 0;
    }

    // Recursive case
    return 1 + countNodes(head.next);
}
```


<Exercises>

Count nodes in a LinkedList

Create the call and explain step by step what is happening during the search

```
public static int countNodes(Node head) {

    // Base case
    if (head == null) {
        return 0;
    }

    // Recursive case
    return 1 + countNodes(head.next);
}
```


MEME

Thank you

Australian National University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C